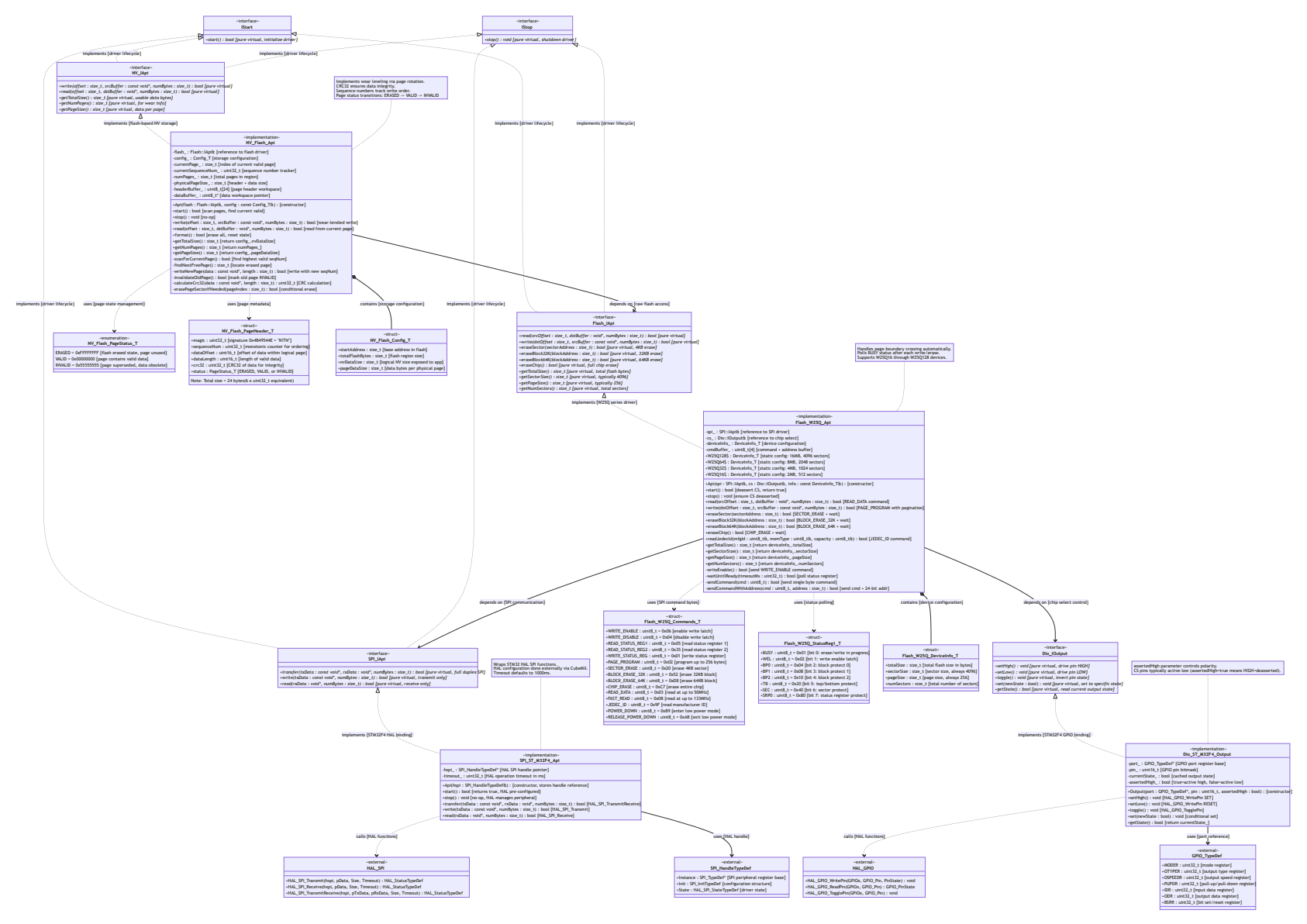
«interface»
IStart
+start() : bool [pure virtual, initialize driver]
«interface»
IStop
+stop() : void [pure virtual, shutdown driver]
implements [driver lifecycle]
implements [driver lifecycle]
«interface»
NV_IApi
+write(offset : size_t, srcBuffer : const void*, numBytes : size_t) : bool [pure virtual]
+read(offset : size_t, dstBuffer : void*, numBytes : size_t) : bool [pure virtual]
+getTotalSize() : size_t [pure virtual, usable data bytes]
+getNumPages() : size_t [pure virtual, for wear info]
+getPageSize() : size_t [pure virtual, data per page]
Implements wear leveling via page rotation.
CRC32 ensures data integrity.
Sequence numbers track write order.
Page status transitions: ERASED -> VALID -> INVALID
implements [flash-based NV storage]
implements [driver lifecycle]
implements [driver lifecycle]
«implementation»
NV_Flash_Api
-flash_ : Flash::IApi& [reference to flash driver]
-config_ : Config_T [storage configuration]
-currentPage_ : size_t [index of current valid page]
-currentSequenceNum_ : uint32_t [sequence number tracker]
-numPages_ : size_t [total pages in region]
-physicalPageSize_ : size_t [header + data size]
-headerBuffer_ : uint8_t[24] [page header workspace]
-dataBuffer_ : uint8_t* [data workspace pointer]
+ApiFlash : Flash::IApi&, config : const Config_T&) : [constructor]
+start() : bool [scan pages, find current valid]
+stop() : void [no-op]
+write(offset : size_t, srcBuffer : const void*, numBytes : size_t) : bool [wear-leveled write]
+read(offset : size_t, dstBuffer : void*, numBytes : size_t) : bool [read from current page]
+format() : bool [erase all, reset state]
+getTotalSize() : size_t [return config_.nvDataSize]
+getNumPages() : size_t [return numPages_]
+getPageSize() : size_t [return config_.pageDataSize]
-scanForCurrentPage() : bool [find highest valid seqNum]
-findNextFreePage() : size_t [locate erased page]
-writeNewPage(data : const void*, length : size_t) : bool [write with new seqNum]
-invalidateOldPages() : bool [mark old page INVALID]
-calculateCrc32(data : const void*, length : size_t) : uint32_t [CRC calculation]
-eraseIfPageSectorOfThreshold(pageIndex : size_t) : bool [conditional erase]
implements [driver lifecycle]
uses [page state management]
uses [page metadata]
contains [storage configuration]
implements [driver lifecycle]
depends on [raw flash access]
«enumeration»
NV_Flash_PageStatus_T
ERASED = 0xFFFFFFFF [flash erased state, page unused]
VALID = 0xDDDDDDDD [page contains valid data]
INVALID = 0x55555555 [page superseded, data obsolete]
«struct»
NV_Flash_PageHeader_T
+magic : uint32_t [signature 0x4B495A4E = 'KIZK']
+sequenceNum : uint32_t [monotonic counter for ordering]
+dataOffset : uint16_t [offset of data within logical page]
+dataLength : uint16_t [length of valid data]
+crc32 : uint32_t [CRC32 of data for integrity]
+status : PageStatus_T [ERASED, VALID, or INVALID]
Note: Total size = 24 bytes(6 x uint32_t equivalent)
«struct»
NV_Flash_Config_T
+startAddress : size_t [base address in flash]
+totalFlashBytes : size_t [flash region size]
+nvDataSize : size_t [logical NV size exposed to app]
+pageDataSize : size_t [data bytes per physical page]
«interface»
Flash_IApi
+read(srcOffset : size_t, dstBuffer : void*, numBytes : size_t) : bool [pure virtual]
+write(dstOffset : size_t, srcBuffer : const void*, numBytes : size_t) : bool [pure virtual]
+eraseSector(sectorAddress : size_t) : bool [pure virtual, 4KB erase]
+eraseBlock32K(blockAddress : size_t) : bool [pure virtual, 32KB erase]
+eraseBlock64K(blockAddress : size_t) : bool [pure virtual, 64KB erase]
+eraseChip() : bool [pure virtual, full chip erase]
+getTotalSize() : size_t [pure virtual, total flash bytes]
+getSectorSize() : size_t [pure virtual, typically 4096]
+getPageSize() : size_t [pure virtual, typically 256]
+getNumSectors() : size_t [pure virtual, total sectors]
Handles page-boundary crossing automatically.
Polls BUSY status after each write/erase.
Supports W25Q16 through W25Q128 devices.
implements [W25Q series driver]
«implementation»
Flash_W25Q_Api
-spi_ : SPI::ISpi& [reference to SPI driver]
-cs_ : Dio::IOutputs& [reference to chip select]
-deviceInfo_ : DeviceInfo_T [device configuration]
-cmdBuffer_ : uint8_t[4] [command + address buffer]
+W25Q128FS : DeviceInfo_T [static config: 16MB, 4096 sectors]
+W25Q64FS : DeviceInfo_T [static config: 8MB, 2048 sectors]
+W25Q32FS : DeviceInfo_T [static config: 4MB, 1024 sectors]
+W25Q16FS : DeviceInfo_T [static config: 2MB, 512 sectors]
+ApiSpi : SPI::ISpi&, cs : Dio::IOutputs&, info : const DeviceInfo_T&) : [constructor]
+start() : bool [deassert CS, return true]
+stop() : void [ensure CS deasserted]
+read(srcOffset : size_t, dstBuffer : void*, numBytes : size_t) : bool [READ_DATA command]
+write(dstOffset : size_t, srcBuffer : const void*, numBytes : size_t) : bool [PAGE_PROGRAM with pagination]
+eraseSector(sectorAddress : size_t) : bool [SECTOR_ERASE + wait]
+eraseBlock32K(blockAddress : size_t) : bool [BLOCK_ERASE_32K + wait]
+eraseBlock64K(blockAddress : size_t) : bool [BLOCK_ERASE_64K + wait]
+eraseChip() : bool [CHIP_ERASE + wait]
+readJedecId(jedecIdOut : uint8_t&, memType : uint8_t&, capacity : uint8_t&) : bool [JEDEC_ID command]
+getTotalSize() : size_t [return deviceInfo_.totalSize]
+getSectorSize() : size_t [return deviceInfo_.sectorSize]
+getPageSize() : size_t [return deviceInfo_.pageSize]
+getNumSectors() : size_t [return deviceInfo_.numSectors]
-writeEnable() : bool [send WRITE_ENABLE command]
-waitUntilReady(timeoutMs : uint32_t) : bool [poll status register]
-sendCommand(cmd : uint8_t) : bool [send single byte command]
-sendCommandWithAddress(cmd : uint8_t, address : size_t) : bool [send cmd + 24-bit addr]
depends on [SPI communication]
uses [SPI command bytes]
uses [status polling]
contains [device configuration]
depends on [chip select control]
«interface»
SPI_IApi
+transfer(txData : const void*, rxData : void*, numBytes : size_t) : bool [pure virtual, full duplex SPI]
+write(txData : const void*, numBytes : size_t) : bool [pure virtual, transmit only]
+read(rxData : void*, numBytes : size_t) : bool [pure virtual, receive only]
Wraps STM32 HAL SPI functions.
HAL configuration done externally via CubeMX.
Timeout defaults to 1000ms.
«struct»
Flash_W25Q_Commands_T
+WRITE_ENABLE : uint8_t = 0x06 [enable write latch]
+WRITE_DISABLE : uint8_t = 0x04 [disable write latch]
+READ_STATUS_REG1 : uint8_t = 0x05 [read status register 1]
+READ_STATUS_REG2 : uint8_t = 0x35 [read status register 2]
+WRITE_STATUS_REG : uint8_t = 0x01 [write status register]
+PAGE_PROGRAM : uint8_t = 0x02 [program up to 256 bytes]
+SECTOR_ERASE : uint8_t = 0x20 [erase 4KB sector]
+BLOCK_ERASE_32K : uint8_t = 0x52 [erase 32KB block]
+BLOCK_ERASE_64K : uint8_t = 0xD8 [erase 64KB block]
+CHIP_ERASE : uint8_t = 0xC7 [erase entire chip]
+READ_DATA : uint8_t = 0x03 [read at up to 50MHz]
+FAST_READ : uint8_t = 0x0B [read at up to 133MHz]
+JEDEC_ID : uint8_t = 0x9F [read manufacturer ID]
+POWER_DOWN : uint8_t = 0xB9 [enter low power mode]
+RELEASE_POWER_DOWN : uint8_t = 0xAB [exit low power mode]
«struct»
Flash_W25Q_StatusReg1_T
+BUSY : uint8_t = 0x01 [bit 0: erase/write in progress]
+WEL : uint8_t = 0x02 [bit 1: write enable latch]
+BP0 : uint8_t = 0x04 [bit 2: block protect 0]
+BP1 : uint8_t = 0x08 [bit 3: block protect 1]
+BP2 : uint8_t = 0x10 [bit 4: block protect 2]
+TB : uint8_t = 0x20 [bit 5: top/bottom protect]
+SEC : uint8_t = 0x40 [bit 6: sector protect]
+SRP0 : uint8_t = 0x80 [bit 7: status register protect]
«struct»
Flash_W25Q_DeviceInfo_T
+totalSize : size_t [total flash size in bytes]
+sectorSize : size_t [sector size, always 4096]
+pageSize : size_t [page size, always 256]
+numSectors : size_t [total number of sectors]
«interface»
Dio_IOutput
+write(high) : void [pure virtual, drive pin HIGH]
+setLow() : void [pure virtual, drive pin LOW]
+toggle() : void [pure virtual, invert pin state]
+set(newState : bool) : void [pure virtual, set to specific state]
-getState() : bool [pure virtual, read current output state]
assertedHigh parameter controls polarity.
CS pins typically active-low (assertedHigh=true means HIGH=deasserted).
implements [STM32F4 HAL binding]
implements [STM32F4 GPIO binding]
«implementation»
SPI_ST_M32F4_Api
-hspi_ : SPI_HandleTypeDef* [HAL SPI handle pointer]
-timeout_ : uint32_t [HAL operation timeout in ms]
+ApiHspi : SPI_HandleTypeDef&) : [constructor, stores handle reference]
+start() : bool [returns true, HAL pre-configured]
+stop() : void [no-op, HAL manages peripheral]
+transfer(txData : const void*, rxData : void*, numBytes : size_t) : bool [HAL_SPI_TransmitReceive]
+write(txData : const void*, numBytes : size_t) : bool [HAL_SPI_Transmit]
+read(rxData : void*, numBytes : size_t) : bool [HAL_SPI_Receive]
«implementation»
Dio_ST_M32F4_Output
-port_ : GPIO_TypeDef* [GPIO port register base]
-pin_ : uint16_t [GPIO pin bitmask]
-currentState_ : bool [cached output state]
-assertedHigh_ : bool [true=active high, false=active low]
+OutputsHport : GPIO_TypeDef*, pin : uint16_t, assertedHigh : bool) : [constructor]
+write(high) : void [HAL_GPIO_WritePin SET]
+setLow() : void [HAL_GPIO_WritePin RESET]
+toggle() : void [HAL_GPIO_TogglePin]
+set(newState : bool) : void [conditional set]
+getState() : bool [return currentState_]
calls [HAL functions]
uses [HAL handle]
calls [HAL functions]
uses [port reference]
«external»
HAL_SPI
+HAL_SPI_Transmit(hspi, pData, Size, Timeout) : HAL_StatusTypeDef
+HAL_SPI_Receive(hspi, pData, Size, Timeout) : HAL_StatusTypeDef
+HAL_SPI_TransmitReceive(hspi, pTxData, pRxData, Size, Timeout) : HAL_StatusTypeDef
«external»
SPI_HandleTypeDef
+Instance : SPI_TypeDef* [SPI peripheral register base]
+Init : SPI_InitTypeDef [configuration structure]
+State : HAL_SPI_StateTypeDef [driver state]
«external»
HAL_GPIO
+HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState) : void
+HAL_GPIO_ReadPin(GPIOx, GPIO_Pin) : GPIO_PinState
+HAL_GPIO_TogglePin(GPIOx, GPIO_Pin) : void
«external»
GPIO_TypeDef
+MODER : uint32_t [mode register]
+OTYPER : uint32_t [output type register]
+OSPEEDR : uint32_t [output speed register]
+PUPDR : uint32_t [pull-up/pull-down register]
+IDR : uint32_t [input data register]
+ODR : uint32_t [output data register]
+BSRR : uint32_t [bit set/reset register]